Exercise sheet 1

Logic Programming (621.315), Winter semester 2026

Raphael Bleier

Demonstration: 21 October 2026

Erklärung

Ein Atom ist hier eine einzelne Aussage, etwa `bird(tweety)`. Ein Modell erfüllt alle Regeln, muss aber nicht die kleinste begründbare Menge sein. Ein Answer Set ist die kleinste abgeschlossene Menge seines eigenen Redukts. Default-Negation `not a` prüft, ob a im Kandidaten fehlt; sie ist keine zusätzliche Aussage $\neg a$. Wir verwenden die Begriffe und die Notation $\mathrm{Cn}(P)$ aus den Kursnotizen zu positiven und normalen Programmen. Ein Punkt beendet jede Regel. Eine Regel wie `kopf :- bedingung1, bedingung2.` leitet den Kopf ab, wenn beide Bedingungen gelten. Das Komma bedeutet UND. Fakten wie `a.` haben keinen Regelkörper. Die Programme liegen zusätzlich als ausführbare `.lp`-Dateien im Unterordner `clingo/`. Mit `-n 0` lässt Clingo alle Answer Sets ausgeben.

Quiz

1 *Each positive logic program has at most one model.*

Take $P = \{a :- b\}$. Both $\varnothing$ and $\{a, b\}$ satisfy the rule; they are distinct models. □

False

Erklärung

Hier unterscheiden wir gewöhnliche Modelle von Answer Sets. Für ein Answer Set dürfen keine Atome ohne eine Herleitung aus dem Redukt übrig bleiben. Die Aussage über normale Programme nutzt ausdrücklich die Kursdefinition mit genau einem Atom im Regelkopf.

2 *Each positive logic program has at most one answer set.*

A positive program has a unique least model, obtained by iterating its consequence operator. Its answer set is this least model. □

True

Erklärung

Hier unterscheiden wir gewöhnliche Modelle von Answer Sets. Für ein Answer Set dürfen keine Atome ohne eine Herleitung aus dem Redukt übrig bleiben. Die Aussage über normale Programme nutzt ausdrücklich die Kursdefinition mit genau einem Atom im Regelkopf.

3 *Each normal logic program has at least one model.*

Under the course definition, every rule has an atom in its head (no headless constraints). The interpretation $A(P)$ containing all atoms satisfies every rule, since every head is true. □

True

Erklärung

Hier unterscheiden wir gewöhnliche Modelle von Answer Sets. Für ein Answer Set dürfen keine Atome ohne eine Herleitung aus dem Redukt übrig bleiben. Die Aussage über normale Programme nutzt ausdrücklich die Kursdefinition mit genau einem Atom im Regelkopf.

4 *Each normal logic program has at least one answer set.*

Take $P = \{a :- not\, a\}$. For $X = \varnothing$, the reduct contains the fact a and has least model $\{a\}$. For $X = \{a\}$, the reduct is empty and has least model $\varnothing$. Neither candidate equals its reduct least model. □

False

Erklärung

Hier unterscheiden wir gewöhnliche Modelle von Answer Sets. Für ein Answer Set dürfen keine Atome ohne eine Herleitung aus dem Redukt übrig bleiben. Die Aussage über normale Programme nutzt ausdrücklich die Kursdefinition mit genau einem Atom im Regelkopf.

Exercise 1.1

Determine the answer set of the following positive logic programs.

(a)

Rule numbers refer to the following order (one rule ends at each period).

```
a.
b :- e.
b :- f.
c :- b, f.
c :- d, g.
d.
e :- b, d.
g :- a, b.
g :- a, d.
```

Erklärung

Die Fakten `a.` und `d.` starten die Ableitung. Die Regel `g :- a, d.` leitet g ab; anschließend leitet `c :- d, g.` das Atom c ab. b und e hängen nur voneinander ab und haben keinen Startfakt. Für f gibt es überhaupt keine Regel mit diesem Kopf, daher wird f nicht hergeleitet.

Erklärung

Das Programm enthält keine Default-Negation. Wir beginnen bei der leeren Menge und wenden nur Regeln an, deren Körper bereits erfüllt ist. Fakten starten die Herleitung. Eine positive Schleife ohne diesen Start liefert dagegen keine Atome.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\texttt{a}\} \qquad \text{fact, rule 1}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `a` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\texttt{d}\} \qquad \text{fact, rule 6}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `d` ist damit ein Fakt und muss in die Menge.

$$M_3 = M_2 \cup \{\texttt{g}\} \qquad \text{rule 9}$$

Erklärung

Für `g` brauchen wir `a` und `d`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_4 = M_3 \cup \{\texttt{c}\} \qquad \text{rule 5}$$

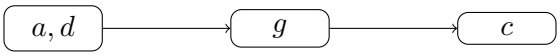
Erklärung

Für `c` brauchen wir `d` und `g`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

No rule has a true body and a head outside M_4; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \{\texttt{a}, \texttt{c}, \texttt{d}, \texttt{g}\}$$

Neither b nor e has an initial derivation; f has no head rule. The rules linking b, e cannot start.



Erklärung

Die Pfeile zeigen den Weg vom Faktpaar zur neuen Aussage. Für g brauchen wir a, d; für c danach noch d, das als Fakt bereits vorhanden ist. Der Kreis zwischen b und e liegt außerhalb dieses Herleitungswegs und bekommt keinen Start.

$$\boxed{X = \{\texttt{a}, \texttt{c}, \texttt{d}, \texttt{g}\}}$$

Erklärung

Die Probe besteht aus zwei Teilen: Jede aufgenommene Aussage hat oben eine passende Regel. Am Ende kann keine Regel ein weiteres Atom hinzufügen. Damit ist die Menge ein Modell und zugleich das kleinste Modell.

(b)

Rule numbers refer to the following order (one rule ends at each period).

```
likes(kate,fish).
likes(kate,soup).
likes(mike,beef).
likes(mike,cake).
likes(paul,duck).
likes(paul,pork).
likes(tina,beef).
likes(tina,soup).
order(beef) :- likes(mike,beef), likes(tina,beef).
order(soup) :- likes(kate,soup), likes(tina,soup).
happy(kate) :- likes(kate,fish), order(fish).
happy(kate) :- likes(kate,soup), order(soup).
happy(mike) :- likes(mike,beef), order(beef).
happy(mike) :- likes(mike,cake), order(cake).
happy(paul) :- likes(paul,duck), order(duck).
happy(paul) :- likes(paul,pork), order(pork).
happy(tina) :- likes(tina,beef), order(beef).
happy(tina) :- likes(tina,soup), order(soup).
```

Erklärung

Die `likes`-Fakten beschreiben, was jede Person gern isst. Eine `order`-Regel feuert erst, wenn beide genannten Personen das Gericht mögen. `happy(X)` folgt anschließend nur, wenn für eine Speise, die X mag, auch eine Bestellung hergeleitet wurde. Deshalb entstehen Bestellungen für Beef und Suppe, aber nicht für Cake, Duck oder Pork.

Erklärung

Das Programm enthält keine Default-Negation. Wir beginnen bei der leeren Menge und wenden nur Regeln an, deren Körper bereits erfüllt ist. Fakten starten die Herleitung. Eine positive Schleife ohne diesen Start liefert dagegen keine Atome.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\texttt{likes(kate,fish)}\} \qquad \text{fact, rule 1}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `likes(kate,fish)` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\texttt{likes(kate,soup)}\} \qquad \text{fact, rule 2}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `likes(kate,soup)` ist damit ein Fakt und muss in die Menge.

$$M_3 = M_2 \cup \{\texttt{likes(mike,beef)}\} \qquad \text{fact, rule 3}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `likes(mike,beef)` ist damit ein Fakt und muss in die Menge.

$$M_4 = M_3 \cup \{\texttt{likes(mike,cake)}\} \qquad \text{fact, rule 4}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `likes(mike,cake)` ist damit ein Fakt und muss in die Menge.

$$M_5 = M_4 \cup \{\texttt{likes(paul,duck)}\} \qquad \text{fact, rule 5}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `likes(paul,duck)` ist damit ein Fakt und muss in die Menge.

$$M_6 = M_5 \cup \{\texttt{likes(paul,pork)}\} \qquad \text{fact, rule 6}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `likes(paul,pork)` ist damit ein Fakt und muss in die Menge.

$$M_7 = M_6 \cup \{\texttt{likes(tina,beef)}\} \qquad \text{fact, rule 7}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `likes(tina,beef)` ist damit ein Fakt und muss in die Menge.

$$M_8 = M_7 \cup \{\texttt{likes(tina,soup)}\} \qquad \text{fact, rule 8}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `likes(tina,soup)` ist damit ein Fakt und muss in die Menge.

$$M_9 = M_8 \cup \{\texttt{order(beef)}\} \qquad \text{rule 9}$$

Erklärung

Für `order(beef)` brauchen wir `likes(mike,beef)` und `likes(tina,beef)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_{10} = M_9 \cup \{\texttt{order(soup)}\} \qquad \text{rule 10}$$

Erklärung

Für `order(soup)` brauchen wir `likes(kate,soup)` und `likes(tina,soup)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_{11} = M_{10} \cup \{\texttt{happy(kate)}\} \qquad \text{rule 12}$$

Erklärung

Für `happy(kate)` brauchen wir `likes(kate,soup)` und `order(soup)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_{12} = M_{11} \cup \{\texttt{happy(mike)}\} \qquad \text{rule 13}$$

Erklärung

Für `happy(mike)` brauchen wir `likes(mike,beef)` und `order(beef)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_{13} = M_{12} \cup \{\texttt{happy(tina)}\} \qquad \text{rule 17}$$

Erklärung

Für `happy(tina)` brauchen wir `likes(tina,beef)` und `order(beef)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

No rule has a true body and a head outside M_{13}; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \left\{ \begin{array}{c} \texttt{happy(kate)},\ \texttt{happy(mike)},\ \texttt{happy(tina)}, \\ \texttt{likes(kate,fish)},\ \texttt{likes(kate,soup)}, \\ \texttt{likes(mike,beef)},\ \texttt{likes(mike,cake)}, \\ \texttt{likes(paul,duck)},\ \texttt{likes(paul,pork)}, \\ \texttt{likes(tina,beef)},\ \texttt{likes(tina,soup)},\ \texttt{order(beef)}, \\ \texttt{order(soup)} \end{array} \right\}$$

$$X = \left\{ \begin{array}{c} \texttt{happy(kate)},\ \texttt{happy(mike)},\ \texttt{happy(tina)}, \\ \texttt{likes(kate,fish)},\ \texttt{likes(kate,soup)}, \\ \texttt{likes(mike,beef)},\ \texttt{likes(mike,cake)}, \\ \texttt{likes(paul,duck)},\ \texttt{likes(paul,pork)}, \\ \texttt{likes(tina,beef)},\ \texttt{likes(tina,soup)},\ \texttt{order(beef)}, \\ \texttt{order(soup)} \end{array} \right\}$$

Erklärung

Die Probe besteht aus zwei Teilen: Jede aufgenommene Aussage hat oben eine passende Regel. Am Ende kann keine Regel ein weiteres Atom hinzufügen. Damit ist die Menge ein Modell und zugleich das kleinste Modell.

Exercise 1.2

Determine the reducts for the given candidate sets and decide which candidates are answer sets.

(a)

Rule numbers refer to the following order (one rule ends at each period).

```
a :- b.
b :- not c.
c :- not b.
d :- a, e.
e.
```

Erklärung

Die Regeln für b und c bilden eine gegenseitige Default-Negation: `b :- not c.` leitet b ab, wenn c fehlt; `c :- not b.` entsprechend c. Sie erzeugen zwei alternative Fälle. `e.` ist ein Fakt. Wenn b gilt, folgt a und danach d; wenn c gilt, bleibt der positive Kreis über a und d ohne Start. Die Kandidaten werden danach mit dem jeweiligen Redukt geprüft.

Candidate X_1

$$X = \{\texttt{a}, \texttt{b}, \texttt{d}, \texttt{e}\}$$

Reduct $R = P^X$: delete a rule if a default-negated atom belongs to X; remove the remaining default negations. Deletion decisions use the order in P; the subsequent derivation uses the order in the displayed reduct R. Rule 2 is retained because $\{\texttt{c}\} \cap X = \varnothing$; its default negations are removed.

Rule 3 is deleted because $\{\texttt{b}\} \subseteq X$.

Rule numbers refer to the following order (one rule ends at each period).

```
a :- b.
b.
d :- a, e.
e.
```

Erklärung

Beim Redukt bleibt die Kandidatenmenge fest. Wir entscheiden zuerst, welche Regeln wegfallen. Erst danach berechnen wir aus den verbleibenden positiven Regeln die kleinste abgeschlossene Menge. Ein Modell allein reicht nicht: Diese Menge muss genau der Kandidat sein.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\texttt{b}\} \qquad \text{fact, rule 2}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `b` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\texttt{e}\} \qquad \text{fact, rule 4}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `e` ist damit ein Fakt und muss in die Menge.

$$M_3 = M_2 \cup \{\texttt{a}\} \qquad \text{rule 1}$$

Erklärung

Für `a` brauchen wir `b`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_4 = M_3 \cup \{\texttt{d}\} \qquad \text{rule 3}$$

Erklärung

Für `d` brauchen wir `a` und `e`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

No rule has a true body and a head outside M_4; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \{\texttt{a},\ \texttt{b},\ \texttt{d},\ \texttt{e}\}$$

$$\mathrm{Cn}(P^X) = X$$

$$\boxed{X \text{ is an answer set.}}$$

Candidate X_2

$$X = \{\texttt{b},\ \texttt{e}\}$$

Reduct $R = P^X$: delete a rule if a default-negated atom belongs to X; remove the remaining default negations. Deletion decisions use the order in P; the subsequent derivation uses the order in the displayed reduct R. Rule 2 is retained because $\{\texttt{c}\} \cap X = \varnothing$; its default negations are removed.

Rule 3 is deleted because $\{\texttt{b}\} \subseteq X$.

Rule numbers refer to the following order (one rule ends at each period).

```
a :- b.
b.
d :- a, e.
e.
```

Erklärung

Beim Redukt bleibt die Kandidatenmenge fest. Wir entscheiden zuerst, welche Regeln wegfallen. Erst danach berechnen wir aus den verbleibenden positiven Regeln die kleinste abgeschlossene Menge. Ein Modell allein reicht nicht: Diese Menge muss genau der Kandidat sein.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\texttt{b}\} \qquad \text{fact, rule 2}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `b` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\texttt{e}\} \qquad \text{fact, rule 4}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `e` ist damit ein Fakt und muss in die Menge.

$$M_3 = M_2 \cup \{\texttt{a}\} \qquad \text{rule 1}$$

Erklärung

Für `a` brauchen wir `b`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_4 = M_3 \cup \{\texttt{d}\} \qquad \text{rule 3}$$

Erklärung

Für `d` brauchen wir `a` und `e`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

No rule has a true body and a head outside M_4; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \{\texttt{a}, \texttt{b}, \texttt{d}, \texttt{e}\}$$

$$\mathrm{Cn}(P^X) \neq X$$

$$\boxed{X \text{ is not an answer set.}}$$

Missing from the candidate: $\{\texttt{a}, \texttt{d}\}$.

(b)

Rule numbers refer to the following order (one rule ends at each period).

```
a :- not b.
a :- d.
b :- not c.
c :- a, not b.
d :- a.
d :- not c, not e.
e :- b, not a.
e :- d.
```

Erklärung

Die Körper mit `not` prüfen, ob ein Atom im Kandidaten fehlt. Die Regel für d gilt nur dann, wenn sowohl c als auch e fehlen. Die Regeln für a, d und e enthalten positive Abhängigkeiten; ein solcher Kreis reicht allein nicht als Begründung. Darum wird jede Kandidatenmenge über ihr eigenes Redukt geprüft.

Candidate X_1

$$X = \{\texttt{a}, \texttt{b}, \texttt{d}, \texttt{e}\}$$

Reduct $R = P^X$: delete a rule if a default-negated atom belongs to X; remove the remaining default negations. Deletion decisions use the order in P; the subsequent derivation uses the order in the displayed reduct R. Rule 1 is deleted because $\{\texttt{b}\} \subseteq X$.

Rule 3 is retained because $\{\texttt{c}\} \cap X = \varnothing$; its default negations are removed.

Rule 4 is deleted because $\{\texttt{b}\} \subseteq X$.

Rule 6 is deleted because $\{\texttt{e}\} \subseteq X$.

Rule 7 is deleted because $\{\texttt{a}\} \subseteq X$.

Rule numbers refer to the following order (one rule ends at each period).

```
a :- d.
b.
d :- a.
e :- d.
```

Erklärung

Beim Redukt bleibt die Kandidatenmenge fest. Wir entscheiden zuerst, welche Regeln wegfallen. Erst danach berechnen wir aus den verbleibenden positiven Regeln die kleinste abgeschlossene Menge. Ein Modell allein reicht nicht: Diese Menge muss genau der Kandidat sein.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\texttt{b}\} \qquad \text{fact, rule 2}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `b` ist damit ein Fakt und muss in die Menge.

No rule has a true body and a head outside M_1; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \{\texttt{b}\}$$

$$\mathrm{Cn}(P^X) \neq X$$

$$\boxed{X \text{ is not an answer set.}}$$

Unsupported candidate atoms: $\{\texttt{a}, \texttt{d}, \texttt{e}\}$.

Candidate X_2

$$X = \{\texttt{b}, \texttt{e}\}$$

Reduct $R = P^X$: delete a rule if a default-negated atom belongs to X; remove the remaining default negations. Deletion decisions use the order in P; the subsequent derivation uses the order in the displayed reduct R. Rule 1 is deleted because $\{\texttt{b}\} \subseteq X$.

Rule 3 is retained because $\{\texttt{c}\} \cap X = \varnothing$; its default negations are removed.

Rule 4 is deleted because $\{\texttt{b}\} \subseteq X$.

Rule 6 is deleted because $\{\texttt{e}\} \subseteq X$.

Rule 7 is retained because $\{\texttt{a}\} \cap X = \varnothing$; its default negations are removed.

Rule numbers refer to the following order (one rule ends at each period).

```
a :- d.
b.
d :- a.
e :- b.
e :- d.
```

Erklärung

Beim Redukt bleibt die Kandidatenmenge fest. Wir entscheiden zuerst, welche Regeln wegfallen. Erst danach berechnen wir aus den verbleibenden positiven Regeln die kleinste abgeschlossene Menge. Ein Modell allein reicht nicht: Diese Menge muss genau der Kandidat sein.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\mathtt{b}\} \qquad \text{fact, rule 2}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `b` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\mathtt{e}\} \qquad \text{rule 4}$$

Erklärung

Für `e` brauchen wir `b`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

No rule has a true body and a head outside M_2; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \{\mathtt{b},\ \mathtt{e}\}$$

$$\mathrm{Cn}(P^X) = X$$

$$\boxed{X \text{ is an answer set.}}$$

Exercise 1.3

Find all answer sets of both programs from Exercise 1.2 without testing all interpretations.

(a)

Erklärung

Hier wird kein neues Programm eingeführt: Für 1.3a verwenden wir genau den Code aus 1.2a, für 1.3b den aus 1.2b. Jetzt suchen wir alle Answer Sets statt nur die vorgegebenen Kandidaten zu prüfen. Die Fallunterscheidung nach b deckt alle Möglichkeiten ab.

The fact forces e. If $b \in X$, the rule for c is deleted, so c has no supporting rule. Thus $c \notin X$. The reduct contains the fact b; a follows from b, and d follows from a, e. This gives $X = \{a, b, d, e\}$. If $b \notin X$, the reduct contains the fact c. The rule for b is then deleted. Neither a nor d can be derived, giving $X = \{c, e\}$. These two cases exhaust membership of b. □

Erklärung

Wir teilen nur nach Atomen auf, die eine echte Entscheidung durch Default-Negation auslösen. Danach erzwingen Fakten und Regeln den Rest. Ein positiver Kreis begründet sich nicht selbst: Ohne Startregel bleibt er beim Aufbau der kleinsten Menge leer.

$$\boxed{\#\,\mathrm{AS}(P) = 2}$$

Answer set 1 and reduct check

$$X = \{\texttt{a},\ \texttt{b},\ \texttt{d},\ \texttt{e}\}$$

Reduct $R = P^X$: delete a rule if a default-negated atom belongs to X; remove the remaining default negations. Deletion decisions use the order in P; the subsequent derivation uses the order in the displayed reduct R. Rule 2 is retained because $\{\texttt{c}\} \cap X = \varnothing$; its default negations are removed.

Rule 3 is deleted because $\{\texttt{b}\} \subseteq X$.

Rule numbers refer to the following order (one rule ends at each period).

```
a :- b.
b.
d :- a, e.
e.
```

Erklärung

Beim Redukt bleibt die Kandidatenmenge fest. Wir entscheiden zuerst, welche Regeln wegfallen. Erst danach berechnen wir aus den verbleibenden positiven Regeln die kleinste abgeschlossene Menge. Ein Modell allein reicht nicht: Diese Menge muss genau der Kandidat sein.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\texttt{b}\} \qquad \text{fact, rule 2}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `b` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\texttt{e}\} \qquad \text{fact, rule 4}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `e` ist damit ein Fakt und muss in die Menge.

$$M_3 = M_2 \cup \{\texttt{a}\} \qquad \text{rule 1}$$

Erklärung

Für `a` brauchen wir `b`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_4 = M_3 \cup \{\texttt{d}\} \qquad \text{rule 3}$$

Erklärung

Für `d` brauchen wir `a` und `e`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

No rule has a true body and a head outside M_4; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \{\texttt{a},\ \texttt{b},\ \texttt{d},\ \texttt{e}\}$$

$$\mathrm{Cn}(P^X) = X$$

$$\boxed{X \text{ is an answer set.}}$$

Answer set 2 and reduct check

$$X = \{\texttt{c},\ \texttt{e}\}$$

Reduct $R = P^X$: delete a rule if a default-negated atom belongs to X; remove the remaining default negations. Deletion decisions use the order in P; the subsequent derivation uses the order in the displayed reduct R. Rule 2 is deleted because $\{\texttt{c}\} \subseteq X$.

Rule 3 is retained because $\{\texttt{b}\} \cap X = \varnothing$; its default negations are removed.

Rule numbers refer to the following order (one rule ends at each period).

```
a :- b.
c.
d :- a, e.
e.
```

Erklärung

Beim Redukt bleibt die Kandidatenmenge fest. Wir entscheiden zuerst, welche Regeln wegfallen. Erst danach berechnen wir aus den verbleibenden positiven Regeln die kleinste abgeschlossene Menge. Ein Modell allein reicht nicht: Diese Menge muss genau der Kandidat sein.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\texttt{c}\} \qquad \text{fact, rule 2}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `c` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\texttt{e}\} \qquad \text{fact, rule 4}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `e` ist damit ein Fakt und muss in die Menge.

No rule has a true body and a head outside M_2; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \{\texttt{c},\ \texttt{e}\}$$

$$\mathrm{Cn}(P^X) = X$$

$$\boxed{X \text{ is an answer set.}}$$

The complete list of answer sets is:

$$\boxed{X_1 = \{\texttt{a},\ \texttt{b},\ \texttt{d},\ \texttt{e}\}}$$

$$\boxed{X_2 = \{\texttt{c},\ \texttt{e}\}}$$

(b)

If $b \notin X$, the reduct contains the fact a. The rules for d, e, and c then derive these three atoms. Since $c \in X$, the rule for b is deleted. The only candidate in this case is $\{a, c, d, e\}$.

If $b \in X$, support for b requires $c \notin X$. Also $e \in X$: if $a \in X$, the rules d from a and e from d force e; if $a \notin X$, the rule e from $b, \mathit{not}\, a$ forces e. Therefore the rule with body $\mathit{not}\, c, \mathit{not}\, e$ is deleted. The remaining rules for a, d form only the positive cycle a from d, d from a, which has no initial fact. Neither atom belongs to the least model. We obtain $\{b, e\}$. These two cases exhaust membership of b. □

Erklärung

Wir teilen nur nach Atomen auf, die eine echte Entscheidung durch Default-Negation auslösen. Danach erzwingen Fakten und Regeln den Rest. Ein positiver Kreis begründet sich nicht selbst: Ohne Startregel bleibt er beim Aufbau der kleinsten Menge leer.

$$\boxed{\#\,\mathrm{AS}(P) = 2}$$

Answer set 1 and reduct check

$$X = \{\mathtt{a},\ \mathtt{c},\ \mathtt{d},\ \mathtt{e}\}$$

Reduct $R = P^X$: delete a rule if a default-negated atom belongs to X; remove the remaining default negations. Deletion decisions use the order in P; the subsequent derivation uses the order in the displayed reduct R. Rule 1 is retained because $\{\mathtt{b}\} \cap X = \varnothing$; its default negations are removed.

Rule 3 is deleted because $\{\mathtt{c}\} \subseteq X$.

Rule 4 is retained because $\{\mathtt{b}\} \cap X = \varnothing$; its default negations are removed.

Rule 6 is deleted because $\{\mathtt{c},\ \mathtt{e}\} \subseteq X$.

Rule 7 is deleted because $\{\mathtt{a}\} \subseteq X$.

Rule numbers refer to the following order (one rule ends at each period).

```
a.
a :- d.
c :- a.
d :- a.
e :- d.
```

Erklärung

Beim Redukt bleibt die Kandidatenmenge fest. Wir entscheiden zuerst, welche Regeln wegfallen. Erst danach berechnen wir aus den verbleibenden positiven Regeln die kleinste abgeschlossene Menge. Ein Modell allein reicht nicht: Diese Menge muss genau der Kandidat sein.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\mathtt{a}\} \qquad \text{fact, rule 1}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `a` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\mathtt{c}\} \qquad \text{rule 3}$$

Erklärung

Für `c` brauchen wir `a`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_3 = M_2 \cup \{\mathtt{d}\} \qquad \text{rule 4}$$

Erklärung

Für `d` brauchen wir `a`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_4 = M_3 \cup \{\texttt{e}\} \qquad \text{rule 5}$$

Erklärung

Für `e` brauchen wir `d`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

No rule has a true body and a head outside M_4; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \{\texttt{a}, \texttt{c}, \texttt{d}, \texttt{e}\}$$

$$\mathrm{Cn}(P^X) = X$$

$$\boxed{X \text{ is an answer set.}}$$

Answer set 2 and reduct check

$$X = \{\texttt{b}, \texttt{e}\}$$

Reduct $R = P^X$: delete a rule if a default-negated atom belongs to X; remove the remaining default negations. Deletion decisions use the order in P; the subsequent derivation uses the order in the displayed reduct R. Rule 1 is deleted because $\{\texttt{b}\} \subseteq X$.

Rule 3 is retained because $\{\texttt{c}\} \cap X = \varnothing$; its default negations are removed.

Rule 4 is deleted because $\{\texttt{b}\} \subseteq X$.

Rule 6 is deleted because $\{\texttt{e}\} \subseteq X$.

Rule 7 is retained because $\{\texttt{a}\} \cap X = \varnothing$; its default negations are removed.

Rule numbers refer to the following order (one rule ends at each period).

```
a :- d.
b.
d :- a.
e :- b.
e :- d.
```

Erklärung

Beim Redukt bleibt die Kandidatenmenge fest. Wir entscheiden zuerst, welche Regeln wegfallen. Erst danach berechnen wir aus den verbleibenden positiven Regeln die kleinste abgeschlossene Menge. Ein Modell allein reicht nicht: Diese Menge muss genau der Kandidat sein.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\texttt{b}\} \qquad \text{fact, rule 2}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `b` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\texttt{e}\} \qquad \text{rule 4}$$

Erklärung

Für `e` brauchen wir `b`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

No rule has a true body and a head outside M_2; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \{\texttt{b},\ \texttt{e}\}$$

$$\mathrm{Cn}(P^X) = X$$

$$\boxed{X \text{ is an answer set.}}$$

The complete list of answer sets is:

$$\boxed{X_1 = \{\texttt{a},\ \texttt{c},\ \texttt{d},\ \texttt{e}\}}$$

$$\boxed{X_2 = \{\texttt{b},\ \texttt{e}\}}$$

Exercise 1.4

Determine all answer sets of the following normal logic programs.

(a)

Rule numbers refer to the following order (one rule ends at each period).

```
canary(tweety).
penguin(tux).
bird(tweety) :- canary(tweety).
bird(tux) :- penguin(tux).
injured(tweety) :- bird(tweety), not ok(tweety).
ok(tweety) :- bird(tweety), not injured(tweety).
injured(tux) :- bird(tux), not ok(tux).
ok(tux) :- bird(tux), not injured(tux).
flies(tweety) :- ok(tweety), not penguin(tweety).
flies(tux) :- ok(tux), not penguin(tux).
```

Erklärung

Die Fakten legen Tweety als Kanarienvogel und Tux als Pinguin fest. Für jeden Vogel wählen die beiden Regeln mit `not injured` und `not ok` genau eine Eigenschaft. Die Flugregel verlangt zusätzlich `not penguin`; deshalb kann Tux auch dann nicht fliegen, wenn `ok(tux)` gilt.

Both bird atoms follow from the facts. For each bird, at least one of `injured` and `ok` must hold: if neither held, the reduct would derive both from the bird atom. They cannot both hold, because then both supporting rules would be deleted. Thus each bird has exactly one of these two properties. The choices are independent. Tweety flies precisely when `ok(tweety)` is chosen, since `penguin(tweety)` has no supporting rule. The rule for Tux flying is always deleted by the penguin fact. The four combinations below exhaust the two independent binary choices. □

Erklärung

Zuerst klären wir, welche Kombinationen überhaupt möglich sind. Gegenseitige Default-Negation lässt genau eine der beiden Eigenschaften zu. Eine Regel mit `not p` und Kopf p kann einen Fall ausschließen, statt einfach p zu einer stabilen Lösung hinzuzufügen. Die

Redukt-Proben darunter prüfen jede verbliebene Kombination einzeln.

$$\boxed{\#\,\mathrm{AS}(P) = 4}$$

Answer set 1 and reduct check

$$X = \left\{ \begin{array}{c} \texttt{bird(tux)},\ \texttt{bird(tweety)},\ \texttt{canary(tweety)}, \\ \texttt{flies(tweety)},\ \texttt{injured(tux)},\ \texttt{ok(tweety)}, \\ \texttt{penguin(tux)} \end{array} \right\}$$

Reduct $R = P^X$: delete a rule if a default-negated atom belongs to X; remove the remaining default negations. Deletion decisions use the order in P; the subsequent derivation uses the order in the displayed reduct R. Rule 5 is deleted because $\{\texttt{ok(tweety)}\} \subseteq X$.

Rule 6 is retained because $\{\texttt{injured(tweety)}\} \cap X = \varnothing$; its default negations are removed.

Rule 7 is retained because $\{\texttt{ok(tux)}\} \cap X = \varnothing$; its default negations are removed.

Rule 8 is deleted because $\{\texttt{injured(tux)}\} \subseteq X$.

Rule 9 is retained because $\{\texttt{penguin(tweety)}\} \cap X = \varnothing$; its default negations are removed.

Rule 10 is deleted because $\{\texttt{penguin(tux)}\} \subseteq X$.

Rule numbers refer to the following order (one rule ends at each period).

```
canary(tweety).
penguin(tux).
bird(tweety) :- canary(tweety).
bird(tux) :- penguin(tux).
ok(tweety) :- bird(tweety).
injured(tux) :- bird(tux).
flies(tweety) :- ok(tweety).
```

Erklärung

Beim Redukt bleibt die Kandidatenmenge fest. Wir entscheiden zuerst, welche Regeln wegfallen. Erst danach berechnen wir aus den verbleibenden positiven Regeln die kleinste abgeschlossene Menge. Ein Modell allein reicht nicht: Diese Menge muss genau der Kandidat sein.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\texttt{canary(tweety)}\} \qquad \text{fact, rule 1}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `canary(tweety)` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\texttt{penguin(tux)}\} \qquad \text{fact, rule 2}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `penguin(tux)` ist damit ein Fakt und muss in die Menge.

$$M_3 = M_2 \cup \{\texttt{bird(tux)}\} \qquad \text{rule 4}$$

Erklärung

Für `bird(tux)` brauchen wir `penguin(tux)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_4 = M_3 \cup \{\texttt{bird(tweety)}\} \qquad \text{rule 3}$$

Erklärung

Für `bird(tweety)` brauchen wir `canary(tweety)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_5 = M_4 \cup \{\texttt{injured(tux)}\} \qquad \text{rule 6}$$

Erklärung

Für `injured(tux)` brauchen wir `bird(tux)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_6 = M_5 \cup \{\texttt{ok(tweety)}\} \qquad \text{rule 5}$$

Erklärung

Für `ok(tweety)` brauchen wir `bird(tweety)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_7 = M_6 \cup \{\texttt{flies(tweety)}\} \qquad \text{rule 7}$$

Erklärung

Für `flies(tweety)` brauchen wir `ok(tweety)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

No rule has a true body and a head outside M_7; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \left\{ \begin{array}{c} \texttt{bird(tux)},\ \texttt{bird(tweety)},\ \texttt{canary(tweety)}, \\ \texttt{flies(tweety)},\ \texttt{injured(tux)},\ \texttt{ok(tweety)}, \\ \texttt{penguin(tux)} \end{array} \right\}$$

$$\mathrm{Cn}(P^X) = X$$

$\boxed{X \text{ is an answer set.}}$

Answer set 2 and reduct check

$$X = \left\{ \begin{array}{c} \texttt{bird(tux)},\ \texttt{bird(tweety)},\ \texttt{canary(tweety)}, \\ \texttt{flies(tweety)},\ \texttt{ok(tux)},\ \texttt{ok(tweety)},\ \texttt{penguin(tux)} \end{array} \right\}$$

Reduct $R = P^X$: delete a rule if a default-negated atom belongs to X; remove the remaining default negations. Deletion decisions use the order in P; the subsequent derivation uses the order in the displayed reduct R. Rule 5 is deleted because $\{\texttt{ok(tweety)}\} \subseteq X$.

Rule 6 is retained because $\{\texttt{injured(tweety)}\} \cap X = \varnothing$; its default negations are removed.

Rule 7 is deleted because $\{\texttt{ok(tux)}\} \subseteq X$.

Rule 8 is retained because $\{\texttt{injured(tux)}\} \cap X = \varnothing$; its default negations are removed.

Rule 9 is retained because $\{\texttt{penguin(tweety)}\} \cap X = \varnothing$; its default negations are removed.

Rule 10 is deleted because $\{\texttt{penguin(tux)}\} \subseteq X$.

Rule numbers refer to the following order (one rule ends at each period).

```
canary(tweety).
penguin(tux).
bird(tweety) :- canary(tweety).
bird(tux) :- penguin(tux).
ok(tweety) :- bird(tweety).
ok(tux) :- bird(tux).
flies(tweety) :- ok(tweety).
```

Erklärung

Beim Redukt bleibt die Kandidatenmenge fest. Wir entscheiden zuerst, welche Regeln wegfallen. Erst danach berechnen wir aus den verbleibenden positiven Regeln die kleinste abgeschlossene Menge. Ein Modell allein reicht nicht: Diese Menge muss genau der Kandidat sein.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\texttt{canary(tweety)}\} \qquad \text{fact, rule 1}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `canary(tweety)` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\texttt{penguin(tux)}\} \qquad \text{fact, rule 2}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `penguin(tux)` ist damit ein Fakt und muss in die Menge.

$$M_3 = M_2 \cup \{\texttt{bird(tux)}\} \qquad \text{rule 4}$$

Erklärung

Für `bird(tux)` brauchen wir `penguin(tux)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_4 = M_3 \cup \{\texttt{bird(tweety)}\} \qquad \text{rule 3}$$

Erklärung

Für `bird(tweety)` brauchen wir `canary(tweety)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_5 = M_4 \cup \{\texttt{ok(tux)}\} \qquad \text{rule 6}$$

Erklärung

Für `ok(tux)` brauchen wir `bird(tux)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_6 = M_5 \cup \{\texttt{ok(tweety)}\} \qquad \text{rule 5}$$

Erklärung

Für `ok(tweety)` brauchen wir `bird(tweety)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_7 = M_6 \cup \{\texttt{flies(tweety)}\} \qquad \text{rule 7}$$

Erklärung

Für `flies(tweety)` brauchen wir `ok(tweety)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

No rule has a true body and a head outside M_7; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \left\{ \begin{array}{c} \texttt{bird(tux)},\ \texttt{bird(tweety)},\ \texttt{canary(tweety)}, \\ \texttt{flies(tweety)},\ \texttt{ok(tux)},\ \texttt{ok(tweety)},\ \texttt{penguin(tux)} \end{array} \right\}$$

$$\mathrm{Cn}(P^X) = X$$

$\boxed{X \text{ is an answer set.}}$

Answer set 3 and reduct check

$$X = \left\{ \begin{array}{c} \texttt{bird(tux)},\ \texttt{bird(tweety)},\ \texttt{canary(tweety)}, \\ \texttt{injured(tux)},\ \texttt{injured(tweety)},\ \texttt{penguin(tux)} \end{array} \right\}$$

Reduct $R = P^X$: delete a rule if a default-negated atom belongs to X; remove the remaining default negations. Deletion decisions use the order in P; the subsequent derivation uses the order in the displayed reduct R. Rule 5 is retained because $\{\texttt{ok(tweety)}\} \cap X = \varnothing$; its default negations are removed.

Rule 6 is deleted because $\{\texttt{injured(tweety)}\} \subseteq X$.

Rule 7 is retained because $\{\texttt{ok(tux)}\} \cap X = \varnothing$; its default negations are removed.

Rule 8 is deleted because $\{\texttt{injured(tux)}\} \subseteq X$.

Rule 9 is retained because $\{\texttt{penguin(tweety)}\} \cap X = \varnothing$; its default negations are removed.

Rule 10 is deleted because $\{\texttt{penguin(tux)}\} \subseteq X$.

Rule numbers refer to the following order (one rule ends at each period).

```
canary(tweety).
penguin(tux).
bird(tweety) :- canary(tweety).
bird(tux) :- penguin(tux).
injured(tweety) :- bird(tweety).
injured(tux) :- bird(tux).
flies(tweety) :- ok(tweety).
```

Erklärung

Beim Redukt bleibt die Kandidatenmenge fest. Wir entscheiden zuerst, welche Regeln wegfallen. Erst danach berechnen wir aus den verbleibenden positiven Regeln die kleinste abgeschlossene Menge. Ein Modell allein reicht nicht: Diese Menge muss genau der Kandidat sein.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\texttt{canary(tweety)}\} \qquad \text{fact, rule 1}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `canary(tweety)` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\texttt{penguin(tux)}\} \qquad \text{fact, rule 2}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `penguin(tux)` ist damit ein Fakt und muss in die Menge.

$$M_3 = M_2 \cup \{\texttt{bird(tux)}\} \qquad \text{rule 4}$$

Erklärung

Für `bird(tux)` brauchen wir `penguin(tux)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_4 = M_3 \cup \{\texttt{bird(tweety)}\} \qquad \text{rule 3}$$

Erklärung

Für `bird(tweety)` brauchen wir `canary(tweety)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_5 = M_4 \cup \{\texttt{injured(tux)}\} \qquad \text{rule 6}$$

Erklärung

Für `injured(tux)` brauchen wir `bird(tux)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer

fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_6 = M_5 \cup \{\texttt{injured(tweety)}\} \qquad \text{rule 5}$$

Erklärung

Für `injured(tweety)` brauchen wir `bird(tweety)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

No rule has a true body and a head outside M_6; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \left\{ \begin{array}{c} \texttt{bird(tux)},\ \texttt{bird(tweety)},\ \texttt{canary(tweety)}, \\ \texttt{injured(tux)},\ \texttt{injured(tweety)},\ \texttt{penguin(tux)} \end{array} \right\}$$

$$\mathrm{Cn}(P^X) = X$$

$\boxed{X \text{ is an answer set.}}$

Answer set 4 and reduct check

$$X = \left\{ \begin{array}{l} \texttt{bird(tux)},\ \texttt{bird(tweety)},\ \texttt{canary(tweety)}, \\ \texttt{injured(tweety)},\ \texttt{ok(tux)},\ \texttt{penguin(tux)} \end{array} \right\}$$

Reduct $R = P^X$: delete a rule if a default-negated atom belongs to X; remove the remaining default negations. Deletion decisions use the order in P; the subsequent derivation uses the order in the displayed reduct R. Rule 5 is retained because $\{\texttt{ok(tweety)}\} \cap X = \varnothing$; its default negations are removed.

Rule 6 is deleted because $\{\texttt{injured(tweety)}\} \subseteq X$.

Rule 7 is deleted because $\{\texttt{ok(tux)}\} \subseteq X$.

Rule 8 is retained because $\{\texttt{injured(tux)}\} \cap X = \varnothing$; its default negations are removed.

Rule 9 is retained because $\{\texttt{penguin(tweety)}\} \cap X = \varnothing$; its default negations are removed.

Rule 10 is deleted because $\{\texttt{penguin(tux)}\} \subseteq X$.

Rule numbers refer to the following order (one rule ends at each period).

```
canary(tweety).
penguin(tux).
bird(tweety) :- canary(tweety).
bird(tux) :- penguin(tux).
injured(tweety) :- bird(tweety).
ok(tux) :- bird(tux).
flies(tweety) :- ok(tweety).
```

Erklärung

Beim Redukt bleibt die Kandidatenmenge fest. Wir entscheiden zuerst, welche Regeln wegfallen. Erst danach berechnen wir aus den verbleibenden positiven Regeln die kleinste abgeschlossene Menge. Ein Modell allein reicht nicht: Diese Menge muss genau der Kandidat sein.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\texttt{canary(tweety)}\} \qquad \text{fact, rule 1}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `canary(tweety)` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\texttt{penguin(tux)}\} \qquad \text{fact, rule 2}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `penguin(tux)` ist damit ein Fakt und muss in die Menge.

$$M_3 = M_2 \cup \{\texttt{bird(tux)}\} \qquad \text{rule 4}$$

Erklärung

Für `bird(tux)` brauchen wir `penguin(tux)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_4 = M_3 \cup \{\texttt{bird(tweety)}\} \qquad \text{rule 3}$$

Erklärung

Für `bird(tweety)` brauchen wir `canary(tweety)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_5 = M_4 \cup \{\texttt{injured(tweety)}\} \qquad \text{rule 5}$$

Erklärung

Für `injured(tweety)` brauchen wir `bird(tweety)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_6 = M_5 \cup \{\texttt{ok(tux)}\} \qquad \text{rule 6}$$

Erklärung

Für `ok(tux)` brauchen wir `bird(tux)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

No rule has a true body and a head outside M_6; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \left\{ \begin{array}{l} \texttt{bird(tux)},\ \texttt{bird(tweety)},\ \texttt{canary(tweety)}, \\ \texttt{injured(tweety)},\ \texttt{ok(tux)},\ \texttt{penguin(tux)} \end{array} \right\}$$

$$\mathrm{Cn}(P^X) = X$$

$$\boxed{X \text{ is an answer set.}}$$

The complete list of answer sets is:

$$\boxed{X_1 = \left\{ \begin{array}{c} \texttt{bird(tux)},\ \texttt{bird(tweety)},\ \texttt{canary(tweety)}, \\ \texttt{flies(tweety)},\ \texttt{injured(tux)},\ \texttt{ok(tweety)}, \\ \texttt{penguin(tux)} \end{array} \right\}}$$

$$\boxed{X_2 = \left\{ \begin{array}{c} \texttt{bird(tux)},\ \texttt{bird(tweety)},\ \texttt{canary(tweety)}, \\ \texttt{flies(tweety)},\ \texttt{ok(tux)},\ \texttt{ok(tweety)},\ \texttt{penguin(tux)} \end{array} \right\}}$$

$$\boxed{X_3 = \left\{ \begin{array}{c} \texttt{bird(tux)},\ \texttt{bird(tweety)},\ \texttt{canary(tweety)}, \\ \texttt{injured(tux)},\ \texttt{injured(tweety)},\ \texttt{penguin(tux)} \end{array} \right\}}$$

$$\boxed{X_4 = \left\{ \begin{array}{c} \texttt{bird(tux)},\ \texttt{bird(tweety)},\ \texttt{canary(tweety)}, \\ \texttt{injured(tweety)},\ \texttt{ok(tux)},\ \texttt{penguin(tux)} \end{array} \right\}}$$

(b)

Rule numbers refer to the following order (one rule ends at each period).

```
num(1).
num(2).
num(3).
lucky(1) :- num(1), not unlucky(1).
unlucky(1) :- num(1), not lucky(1).
lucky(2) :- num(2), not unlucky(2).
unlucky(2) :- num(2), not lucky(2).
lucky(3) :- num(3), not unlucky(3).
unlucky(3) :- num(3), not lucky(3).
p :- unlucky(1), not p.
p :- unlucky(2), not p.
p :- unlucky(3), not p.
```

Erklärung

Die `num`-Fakten liefern die drei Zahlen. Für jede davon schließen sich `lucky` und `unlucky` durch gegenseitige Default-Negation aus. Die drei Regeln für p verwerfen jede Kombination mit einem `unlucky`: Ohne p würde dann p hergeleitet. Mit p wären seine eigenen Regeln gelöscht, sodass p unbegründet bliebe. Übrig bleibt nur der Fall, in dem alle drei Zahlen `lucky` sind.

For each number, exactly one of `lucky` and `unlucky` holds: neither would make both reduct rules fire, while both would delete both supporting rules. Suppose $p \in X$. Every rule with head p is deleted by p, so the least reduct model does not contain p, a contradiction. Hence $p \notin X$. Each remaining rule for p now forbids its unlucky atom: an unlucky atom would derive p in the reduct. All three numbers must therefore be lucky. There is only one candidate. □

Erklärung

Zuerst klären wir, welche Kombinationen überhaupt möglich sind. Gegenseitige Default-Negation lässt genau eine der beiden Eigenschaften zu. Eine Regel mit `not p` und Kopf p kann einen Fall ausschließen, statt einfach p zu einer stabilen Lösung hinzuzufügen. Die Redukt-Proben darunter prüfen jede verbliebene Kombination einzeln.

$$\boxed{\#\,\mathrm{AS}(P) = 1}$$

Answer set 1 and reduct check

$$X = \left\{ \begin{array}{c} \texttt{lucky(1)},\ \texttt{lucky(2)},\ \texttt{lucky(3)},\ \texttt{num(1)},\ \texttt{num(2)}, \\ \texttt{num(3)} \end{array} \right\}$$

Reduct $R = P^X$: delete a rule if a default-negated atom belongs to X; remove the remaining default negations. Deletion decisions use the order in P; the subsequent derivation uses the order in the displayed reduct R. Rule 4 is retained because $\{\texttt{unlucky(1)}\} \cap X = \varnothing$; its default negations are removed.

Rule 5 is deleted because $\{\texttt{lucky(1)}\} \subseteq X$.

Rule 6 is retained because $\{\texttt{unlucky(2)}\} \cap X = \varnothing$; its default negations are removed.

Rule 7 is deleted because $\{\texttt{lucky(2)}\} \subseteq X$.

Rule 8 is retained because $\{\texttt{unlucky(3)}\} \cap X = \varnothing$; its default negations are removed.

Rule 9 is deleted because $\{\texttt{lucky(3)}\} \subseteq X$.

Rule 10 is retained because $\{\texttt{p}\} \cap X = \varnothing$; its default negations are removed.

Rule 11 is retained because $\{\texttt{p}\} \cap X = \varnothing$; its default negations are removed.

Rule 12 is retained because $\{\texttt{p}\} \cap X = \varnothing$; its default negations are removed.

Rule numbers refer to the following order (one rule ends at each period).

```
num(1).
num(2).
num(3).
lucky(1) :- num(1).
lucky(2) :- num(2).
lucky(3) :- num(3).
p :- unlucky(1).
p :- unlucky(2).
p :- unlucky(3).
```

Erklärung

Beim Redukt bleibt die Kandidatenmenge fest. Wir entscheiden zuerst, welche Regeln wegfallen. Erst danach berechnen wir aus den verbleibenden positiven Regeln die kleinste abgeschlossene Menge. Ein Modell allein reicht nicht: Diese Menge muss genau der Kandidat sein.

$$M_0 = \varnothing$$

$$M_1 = M_0 \cup \{\texttt{num(1)}\} \qquad \text{fact, rule 1}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `num(1)` ist damit ein Fakt und muss in die Menge.

$$M_2 = M_1 \cup \{\texttt{num(2)}\} \qquad \text{fact, rule 2}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `num(2)` ist damit ein Fakt und muss in die Menge.

$$M_3 = M_2 \cup \{\texttt{num(3)}\} \qquad \text{fact, rule 3}$$

Erklärung

Diese Regel hat keinen Körper. Ihr Kopf `num(3)` ist damit ein Fakt und muss in die Menge.

$$M_4 = M_3 \cup \{\texttt{lucky(1)}\} \qquad \text{rule 4}$$

Erklärung

Für `lucky(1)` brauchen wir `num(1)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_5 = M_4 \cup \{\texttt{lucky(2)}\} \qquad \text{rule 5}$$

Erklärung

Für `lucky(2)` brauchen wir `num(2)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

$$M_6 = M_5 \cup \{\texttt{lucky(3)}\} \qquad \text{rule 6}$$

Erklärung

Für `lucky(3)` brauchen wir `num(3)`. Weil diese Voraussetzungen bereits hergeleitet sind, dürfen wir den Regelkopf jetzt aufnehmen und alle bisherigen Atome behalten; bei einer fehlenden Voraussetzung würde diese Regel dagegen noch nichts beitragen.

No rule has a true body and a head outside M_6; hence this is the least fixed point.

$$\mathrm{Cn}(R) = \left\{ \begin{matrix} \texttt{lucky(1)},\ \texttt{lucky(2)},\ \texttt{lucky(3)},\ \texttt{num(1)},\ \texttt{num(2)}, \\ \texttt{num(3)} \end{matrix} \right\}$$

$$\mathrm{Cn}(P^X) = X$$

$$\boxed{X \text{ is an answer set.}}$$

The complete list of answer sets is:

$$\boxed{X_1 = \left\{ \begin{matrix} \texttt{lucky(1)},\ \texttt{lucky(2)},\ \texttt{lucky(3)},\ \texttt{num(1)},\ \texttt{num(2)}, \\ \texttt{num(3)} \end{matrix} \right\}}$$